



Lab Assignment #02

OOP

Instructor: Mr. Mohsin Ahmed

Student Name:	Muhammad Abdullah
Registration No:	FA24-BSE-042
Department:	Software Engineering
Submission Date:	19th October 2022

COMSATS University Islamabad

```
package Lab_Assignment_2;

import java.math.BigDecimal;
import java.time.Instant;
import java.util.UUID;

// Interface for refunds
interface Refundable {
    void refund(double amount);
}

// Composition
class AuditService {
    public void record(String message) {
        System.out.println("AUDIT: " + message);
    }
}

// Base class
abstract class Payment {
    // Encapsulation
    private String payerId;
    private double amount;

    // Composition: has-a relationship with AuditService
    protected AuditService auditService;

    // Constructor
    public Payment(String payerId, double amount, AuditService auditService) {
        this.payerId = payerId;
        this.amount = amount;
        this.auditService = auditService;
    }

    // Getter methods
    public String getPayerId() {
        return payerId;
    }

    public double getAmount() {
        return amount;
    }

    public double subtractAmount(double money) {
        return this.amount -= money;
    }

    // Common method
    public void evaluateRisk() {
```

```

        System.out.println("Performing risk evaluation for amount $" + this.amount);
    }

    // Abstract method
    public abstract void processPayment();
}

// Credit Card Payment
class creditCardPayment extends Payment implements Refundable { // implements means agreeing to the contract of
that
                                // interface

    private String cardNumber;

    public creditCardPayment(String payerId, double amount, String cardNumber, AuditService auditService) {
        super(payerId, amount, auditService); // this super() calls the parent class
        this.cardNumber = cardNumber;
    }

    @Override
    public void processPayment() {
        evaluateRisk();
        auditService.record("Processing Credit Card payment for " + getPayerId());
        System.out.println("Payment of $" + getAmount() + " made using Credit Card ending with "
            + cardNumber.substring(cardNumber.length() - 4));
    }

    @Override
    public void refund(double amount) {
        super.subtractAmount(amount);
        System.out.println("Refund of $" + amount + " issued to Credit Card ending with "
            + cardNumber.substring(cardNumber.length() - 4));
    }
}

// Wallet Payment
class digitalWalletPayment extends Payment implements Refundable {
    private String walletId; // encapsulated

    public digitalWalletPayment(String payerId, double amount, String walletId, AuditService auditService) {
        super(payerId, amount, auditService);
        this.walletId = walletId;
    }

    @Override
    public void processPayment() {
        evaluateRisk();
        auditService.record("Processing Digital Wallet payment for " + getPayerId());
    }
}

```

```

        System.out.println("Payment of $" + getAmount() + " completed using Wallet ID: " + walletId);
    }

    @Override
    public void refund(double amount) {
        super.subtractAmount(amount);
        System.out.println("Refund of $" + amount + " issued to Digital Wallet ending with "
            + walletId.substring(walletId.length() - 4));
    }
}

// Crypto Payment
class cryptoCurrencyPayment extends Payment implements Refundable {
    private String blockchainKey; // encapsulated

    public cryptoCurrencyPayment(String payerId, double amount, String blockchainKey, AuditService auditService) {
        super(payerId, amount, auditService);
        this.blockchainKey = blockchainKey;
    }

    @Override
    public void processPayment() {
        evaluateRisk();
        auditService.record("Processing Crypto payment for " + getPayerId());
        System.out
            .println("Crypto transaction of $" + getAmount() + " verified using blockchain key: " + blockchainKey);
    }

    @Override
    public void refund(double amount) {
        super.subtractAmount(amount);
        System.out.println("Refund of $" + amount + " issued to Crypto Wallet ending with "
            + blockchainKey.substring(blockchainKey.length() - 4));
    }
}

public class runner {
    public static void main(String[] args) {
        // One AuditService shared by all ( Composition )
        AuditService audit = new AuditService();

        Payment card = new creditCardPayment("User123", 250.0, "1234567890123456", audit);
        Payment wallet = new digitalWalletPayment("User456", 80.0, "WALLET-9988", audit);
        Payment crypto = new cryptoCurrencyPayment("User789", 500.0, "11111111112222", audit);

        card.processPayment();
        wallet.processPayment();
        crypto.processPayment();
    }
}

```

```

// interface usage
if (card instanceof Refundable) {
    ((Refundable) card).refund(100.0);
}
if (wallet instanceof Refundable) {
    ((Refundable) wallet).refund(50.0);
}

System.out.println(card.getAmount()); // Should get 150
}
}

```

Output:

```

● mabdullah@MacBook Lab_Assignment_2 % java runner.java
Performing risk evaluation for amount $250.0
AUDIT: Processing Credit Card payment for User123
Payment of $250.0 made using Credit Card ending with 3456
Performing risk evaluation for amount $80.0
AUDIT: Processing Digital Wallet payment for User456
Payment of $80.0 completed using Wallet ID: WALLET-9988
Performing risk evaluation for amount $500.0
AUDIT: Processing Crypto payment for User789
Crypto transaction of $500.0 verified using blockchain key: 11111111112222
Refund of $100.0 issued to Credit Card ending with 3456
Refund of $50.0 issued to Digital Wallet ending with 9988
150.0

```

Explanation by concept

1. Inheritance & Method Overriding

- Payment is the **base class**.
- creditCardPayment, digitalWalletPayment, and cryptoCurrencyPayment **inherit** from it.
- Each subclass **overrides** processPayment() to define its own payment logic.

@Override

```
public void processPayment() {  
    // custom logic for each type  
}
```

This shows how **inheritance**, while **overriding** gives each payment type unique behavior.

2. Encapsulation (Data Protection)

- Sensitive fields like cardNumber, walletId, and blockchainKey are **private**.
- They can't be accessed directly from outside the class.
- Only controlled access through methods (getPayerId(), getAmount()) or used internally.

This secures private information (card numbers, wallet IDs, etc.).

3. Composition (Has-A relationship)

- Each Payment **has an** AuditService object.
- Instead of inheriting, it **uses** that service → reduces coupling.

```
protected AuditService auditService; // composition
```

Composition keeps auditing logic separate from payment logic; modular and reusable

4. Extending with Refunds

If in the future more payment types need refunds:

- Use an **interface (Refundable)**, not inheritance.
- Reason: not all payment types will support refunds.
- Using an interface allows **only those that need it** to implement it.

This is more flexible and avoids forcing unnecessary refund behavior into every payment class.